

Intermediate (swap in a structured parser) — Hints

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (proper Python), **Advanced** (what to do when the mismatch happens, not just detect it). Read Basic first even if you already know Python — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — The idea, and the exact pieces](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — The idea, and the exact pieces

Basic Version

You're going to swap the last piece of the chain: instead of "just give me text back," you'll say "give me back an object that matches this exact shape."

To describe the shape, you write a small Pydantic class with named fields, like `name` and `age` . Then you tell your model, "always answer in this shape."

Once that's set up, try asking it something that clearly matches the shape, and something that clearly doesn't — and watch what happens differently.

- A Pydantic model: `from pydantic import BaseModel` then `class Person(BaseModel):`
`name: str; age: int`
- Attach it to the model: `structured_model =`
`ChatOpenAI().with_structured_output(Person)`
- Run it: `structured_model.invoke("...")`
- Catch a mismatch: wrap the call in `try/except Exception as e:` and print `type(e)` .

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

`.with_structured_output(YourModel)` is a method on a chat model that returns a *new* runnable — one whose output is already parsed and validated into `YourModel`, instead of raw text. This is different from manually chaining a `PydanticOutputParser` after the model:

`.with_structured_output(...)` uses the provider's own structured-output feature under the hood when it's available, which is more reliable than asking the model nicely in the prompt and hoping the reply parses.

The key thing to observe: when the model's answer genuinely can't be coerced into your Pydantic shape, the parsing step raises a validation error — and that happens *after* the model has already replied, which is different from Doc04's `response_format`, where the shape is enforced *during* generation.

You can still put this inside a full LCEL chain — `prompt | ChatOpenAI().with_structured_output(Person)` — the parsing step is now built into the model piece itself, so you don't need a separate parser step after it.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Catching the mismatch and printing an error is the minimum — but think about what your *caller* actually wants when this happens. A structured-extraction feature usually can't just crash and stop; it needs to do *something* with an input the model couldn't cleanly shape. The real design question: **should a failed structured call retry with better instructions, fall back to something simpler, or just report failure clearly?** All 3 are legitimate, and picking the wrong one for your situation is a real design mistake, not a style preference.

LangChain gives you a direct tool for the "fall back to something simpler" option:

`.with_fallbacks([...])`. You attach one or more backup `Runnable`s to try, in order, if the first one raises.

```
from langchain_core.output_parsers import StrOutputParser

primary = ChatOpenAI().with_structured_output(Person)
fallback = ChatOpenAI() | StrOutputParser() # plain text, never raises on shape

safe_model = primary.with_fallbacks([fallback])
```

Calling `safe_model.invoke(...)` on a matching input still returns a real `Person`. On a mismatching input, instead of raising, it silently runs the fallback and returns plain text instead — which means your caller now has to check **which kind of result it actually got back** (a `Person` object, or a string), since the type of the result is no longer guaranteed.

Sketch the `isinstance(result, Person)` check a caller would need after using `safe_model` before checking Hint 2.

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate both detect a mismatch and print that it happened. Advanced asks what a real caller should *do* about it, and shows one concrete answer — `.with_fallbacks([...])` — that keeps the chain from raising at all, at the cost of the caller now needing to check what type of result actually came back, since it's no longer guaranteed to be your Pydantic model.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
define a shape called Person with fields: name (text), age (number)

build a model that always answers in the Person shape

test 1: ask it about someone with a clear name and age
    -> print the result, confirm it's a real Person, not text

test 2: ask it something that doesn't have a name and age in it
    -> wrap in try/except, print what kind of error happens
```

Here is almost the whole thing:

```
from pydantic import BaseModel
from langchain_openai import ChatOpenAI

class Person(BaseModel):
    name: str
    age: int

structured_model = ChatOpenAI().with_structured_output(Person)
```

Expected output if you run just this: nothing yet — add the 2 test calls (one printed directly, one inside `try/except`) below it.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
class Person(BaseModel):
    name: str
    age: int

structured_model = ChatOpenAI(model="...", temperature=0).with_structured_output(

good_result = structured_model.invoke("Extract: Maria is 34 years old.")
print(good_result)                                # a real Person object

try:
    bad_result = structured_model.invoke("Extract: the sky is blue.")
except Exception as e:
    print(f"Caught {type(e).__name__}: {e}")
```

Notice you're printing `type(e).__name__`, not just the message — knowing the *exact* exception class is what tells your production code which `except` clause needs to catch it. Write this out fully, run it, and write down the exact class name you actually get before checking Hint 3.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

```

build the primary structured model (as in Intermediate)
build a plain fallback: model | StrOutputParser()

safe_model = primary.with_fallbacks([fallback])

test 1: matching input -> should get a real Person back
test 2: mismatching input -> should get a plain string back, not raise

for each result:
    if isinstance(result, Person): treat as structured
    else: treat as plain text, log that the fallback ran

```

Turning that into real code — fill in the missing piece yourself:

```

from pydantic import BaseModel
from langchain_openai import ChatOpenAI
from langchain_core.output_parsers import StrOutputParser

class Person(BaseModel):
    name: str
    age: int

model = ChatOpenAI(model="gpt-4o-mini", temperature=0)
primary = model.with_structured_output(Person)
fallback = model | StrOutputParser()
safe_model = primary.with_fallbacks([fallback])

good_result = safe_model.invoke("Extract: Maria is 34 years old.")
bad_result = safe_model.invoke("Extract: the sky is blue.")

# your turn: for each of good_result and bad_result, check whether
# it's a Person (isinstance) or plain text, and print which one it is
...

```

Expected output: something showing `good_result` really is a `Person` instance, and `bad_result` is a plain string — neither call raises.

Fill in the `isinstance` check yourself, then compare all 3 of your finished versions against the [Solution](#).

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate both build one structured model, and catch (or observe) the raised error on a mismatch. Advanced attaches a fallback so the chain never raises at all on a mismatch — and shows the real cost of that convenience: the caller now has to check what type actually came back, since "always a `Person`" is no longer a guarantee once a fallback is in play.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)